

PRD: Team Developer Dashboard (GitHub-Integrated)

Status: Draft v1 | **Type:** Product Requirements Document | **Audience:** Engineering, Design, Instructors/Team Leads

1. Overview

The Team Developer Dashboard is a customizable web app that connects to the GitHub API and gives engineering groups one place to see **who is contributing what, what is due when, and which branches are active**. It is built for group projects (student capstones, hackathon teams, small product squads) where work is spread across several repositories and contributors, and where leads need visibility without chasing people for status.

2. Problem Statement

GitHub shows activity per repository, per page. Team leads and members must open many tabs to answer basic questions:

- Who has been contributing, and is the workload balanced?
- Which branches are active, stale, or about to conflict?
- Which milestones and deadlines are at risk?
- Which pull requests are waiting on review?

Existing tools either require heavy setup, cost per seat, or are built for large enterprises rather than small project teams.

3. Goals and Non-Goals

Goals

1. Give a team a single, at-a-glance view of contributions, deadlines, and branches across one or more repos.
2. Let each team customize the dashboard layout, widgets, and tracked projects.
3. Surface risks early (stale branches, overdue milestones, unreviewed PRs) rather than only reporting history.

4. Set up in under 5 minutes via GitHub sign-in and repo selection.

Non-Goals (v1)

- Replacing GitHub Issues/Projects as a task manager.
- Performance evaluation or ranking of individuals for HR purposes.
- Support for GitLab, Bitbucket, or other hosts (considered for v2).
- Code-quality analysis or CI authoring.

4. Target Users

Persona	Needs
Team Lead / Project Manager	Overall health, deadline risk, workload balance, blockers
Developer / Team Member	My open PRs, reviews requested of me, my branches, upcoming deadlines
Instructor / Mentor (view-only)	Progress across multiple teams, evidence of equitable participation

5. User Stories

- As a lead, I can see commits, PRs, and reviews per member over a selectable time range so I can spot imbalance early.
- As a lead, I can define project deadlines and see which are at risk based on open work in the linked milestone.
- As a developer, I can see all active branches, their last activity, ahead/behind status, and linked PRs.
- As a developer, I can see PRs awaiting my review at the top of my view.
- As a team, we can add, remove, and rearrange widgets and save the layout.
- As an instructor, I can view several team dashboards in one read-only overview.
- As an admin, I can control which repos and members are included.

6. Functional Requirements

6.1 Authentication and Setup

- **FR-1:** Sign in with GitHub (OAuth or GitHub App user authorization).
- **FR-2:** Install a GitHub App on a user or organization; select which repositories to track.
- **FR-3:** Create a *Team* with named members mapped to GitHub usernames, and a *Project* grouping one or more repos.

6.2 Contribution Tracking

- **FR-4:** Show commits, PRs opened/merged, reviews given, and issues closed per member, filterable by date range (7/30/90 days, sprint, custom).
- **FR-5:** Display lines added/removed and files changed as secondary metrics, clearly labeled as indicative only.
- **FR-6:** Contribution heatmap (calendar) per member and for the team.
- **FR-7:** Workload balance view showing each member's share of activity, with a configurable threshold for highlighting imbalance.
- **FR-8:** Exclude bot accounts and merge commits by default, with a toggle.

6.3 Deadlines and Milestones

- **FR-9:** Import GitHub milestones and their due dates; allow manually created deadlines in the dashboard.
- **FR-10:** Show progress per milestone (open vs closed issues/PRs) and a countdown.
- **FR-11:** Risk indicator (On track / At risk / Overdue) computed from remaining work, recent velocity, and time left. The formula is visible to users.
- **FR-12:** Timeline (Gantt-style) view of deadlines across projects.
- **FR-13:** Optional reminders via email or Slack webhook (for example, 7 days and 1 day before a deadline).

6.4 Branch and PR Monitoring

- **FR-14:** List all branches with author, last commit time, ahead/behind versus default branch, and linked PR.
- **FR-15:** Flag branches as *Active*, *Stale* (no activity for N days, configurable), or *Merged/Safe to delete*.

- **FR-16:** PR board with status (draft, open, in review, approved, changes requested, merged), age, size, and CI check state.
- **FR-17:** "Needs attention" panel: PRs awaiting review longer than a threshold, failing checks, and merge-conflicted PRs.

6.5 Customization

- **FR-18:** Drag-and-drop widget grid; widgets can be added, removed, resized, and reordered.
- **FR-19:** Widget library for v1: Contribution Summary, Heatmap, Branch List, PR Board, Milestone Progress, Timeline, Needs Attention, Review Queue, Recent Activity Feed.
- **FR-20:** Saved views (for example, "Lead view", "My work") per user, plus a shared team default.
- **FR-21:** Light and dark themes; configurable stale thresholds, metric definitions, and time zone.
- **FR-22:** Export a view as PDF/PNG and data as CSV.

6.6 Roles and Permissions

- **FR-23:** Roles: Owner, Lead, Member, Viewer. Dashboards only show repos the viewing user can access on GitHub.

7. GitHub API Integration

Concern	Approach
Auth	GitHub App with fine-grained permissions (read-only: contents metadata, pull requests, issues, checks, members)
Data fetching	GraphQL API for aggregated queries (PRs, reviews, branches); REST for stats endpoints, compare (ahead/behind), and checks
Freshness	Webhooks (push, pull_request, pull_request_review, issues, milestone, create/delete) update the store in near real time

Concern	Approach
Backfill	Initial historical sync on repo connection, run as a background job with progress indicator
Rate limits	Conditional requests (ETag), caching, request batching, and backoff; surface a "data may be delayed" notice if throttled
Consistency	Nightly reconciliation job to correct missed webhooks

Data model (core entities): User, Team, Membership, Project, Repository, Branch, Commit (aggregated), PullRequest, Review, Issue, Milestone, Deadline, Widget, SavedView.

8. Non-Functional Requirements

- **Performance:** Dashboard loads in under 2 seconds (p95) from cached data; webhook-to-UI latency under 30 seconds.
- **Scale (v1 target):** Up to 50 teams, 20 members per team, 10 repos per project, 2 years of history.
- **Security:** Least-privilege scopes, encrypted tokens at rest, no source code stored (metadata only), audit log of settings changes.
- **Privacy:** Members can see exactly what data is collected about them; data is deleted within 30 days of repo disconnect or account deletion.
- **Reliability:** 99.5% availability; graceful degradation to last cached data when GitHub is unavailable.
- **Accessibility:** WCAG 2.1 AA; charts have text alternatives; color is never the only signal.
- **Responsive:** Usable on tablet and mobile in read-only mode.

9. Responsible Metrics Principles

Activity metrics are imperfect proxies for contribution. The product must:

- Label lines of code and commit counts as indicative, never as a score.
- Avoid single composite "productivity" rankings in v1.
- Credit non-code work where GitHub exposes it (reviews, issue triage, docs changes).

- Give members visibility into the same data their lead sees.

10. Success Metrics

Metric	Target (90 days post-launch)
Setup completion rate (sign-in to first populated dashboard)	80%
Median time to first dashboard	Under 5 minutes
Weekly active teams / teams created	60%
Stale-branch and overdue-PR reduction among active teams	25% decrease
Deadline-risk alerts acted on within 48 hours	50%
User satisfaction (CSAT)	4.2 / 5

11. Suggested Tech Stack (non-binding)

- **Frontend:** React + TypeScript, a grid layout library for drag-and-drop, a charting library (for example, Recharts or D3)
- **Backend:** Node.js/TypeScript API, job queue for syncs, webhook receiver
- **Storage:** PostgreSQL for entities and aggregates, Redis for caching and rate-limit state
- **Hosting:** Containerized deployment with managed Postgres; secrets in a managed vault

12. Milestones

Phase	Scope	Duration
M0: Discovery	Finalize requirements, wireframes, API spike, permission model	2 weeks
M1: Foundation	GitHub App auth, team/project setup, repo sync, data model	3 weeks

Phase	Scope	Duration
M2: Core Views	Contribution summary, heatmap, branch list, PR board	4 weeks
M3: Deadlines	Milestone import, manual deadlines, risk indicator, timeline	3 weeks
M4: Customization	Widget grid, saved views, themes, exports	3 weeks
M5: Beta	Webhook hardening, alerts, accessibility pass, pilot with 3 to 5 teams	3 weeks
M6: Launch	Fixes from beta, docs, onboarding flow, public release	2 weeks

13. Risks and Mitigations

Risk	Impact	Mitigation
GitHub API rate limits on large backfills	Slow or incomplete data	GraphQL batching, ETags, staged backfill, webhook-first updates
Misuse of metrics for unfair evaluation	Trust and adoption loss	Responsible-metrics principles, transparency, no ranking in v1
Members map to multiple GitHub accounts or emails	Inaccurate attribution	Manual identity mapping and merging in team settings
Missed or duplicated webhooks	Stale dashboards	Idempotent handlers, nightly reconciliation
Over-customization complicates UX	Setup friction	Opinionated defaults and a one-click "Recommended layout"
Private-repo data exposure	Security incident	Per-request permission checks against GitHub access, least-privilege scopes

14. Open Questions

1. Will teams primarily be students, professional squads, or both? This affects permissions and instructor features.
2. Is a hosted SaaS or a self-hosted option preferred?
3. Should deadlines sync back to GitHub milestones (two-way) or stay read-only?
4. Is Slack/email alerting required at launch or acceptable for v1.1?
5. Which sprint definition should drive time ranges: calendar weeks, custom sprints, or GitHub Projects iterations?

15. Future Considerations (v2+)

- GitLab and Bitbucket support
- GitHub Projects (v2) board integration and iteration tracking
- CI/CD health and deployment frequency widgets (DORA-style metrics)
- Deeper AI features beyond the v1 Groq-powered summaries (section 16)
- Cross-team benchmarking for instructors and engineering managers

16. AI Features (Groq API)

The dashboard uses the Groq API for fast, low-cost language-model features that explain the data already on screen; it never replaces the underlying numbers.

Features

Feature	What it does	Input sent to Groq
Weekly team summary	Plain-language recap of activity, merged PRs, and deadline status	Aggregated counts, PR titles, milestone names
Risk narrative	Explains why a deadline is marked At risk and suggests next steps	Open/closed counts, velocity, days remaining
PR digest	One-paragraph summary of a PR for reviewers	PR title, description, changed-file names

Feature	What it does	Input sent to Groq
Standup draft	Per-member "yesterday / today / blockers" draft from their activity	That member's own PRs, commits messages, reviews
Natural-language query	"Which branches are stale?" answered by mapping to dashboard filters	The question text and available filter schema

Requirements

- **AI-1:** Call Groq's OpenAI-compatible chat completions endpoint from the backend only; the API key is stored server-side and never reaches the browser.
- **AI-2:** Send metadata only (titles, counts, dates, usernames). Source code and file contents are never sent.
- **AI-3:** AI features are opt-in per team, with a visible toggle and a note on what data is sent.
- **AI-4:** All generated text is labeled "AI-generated" and shows the date range and data it was based on.
- **AI-5:** Use structured (JSON) output for the risk narrative and natural-language query so responses are validated before display.
- **AI-6:** Cache outputs per team and time range; regenerate on demand or when underlying data changes materially.
- **AI-7:** Handle Groq rate limits and errors with retry and backoff; if unavailable, hide the AI panel and show the standard dashboard without error noise.
- **AI-8:** Make the model configurable in settings, so it can change as Groq's model lineup changes.
- **AI-9:** AI output follows the responsible-metrics principles in section 9: no ranking of individuals and no scoring of performance.

Open Questions for AI

1. Which Groq model should be the default, balancing speed, cost, and summary quality?
2. Should teams be able to bring their own Groq API key, or is a single shared key managed by the product?
3. Do instructors need AI summaries across multiple teams in v1, or only per-team?